

Lab 5.4. Use RecyclerView to display a scrollable list

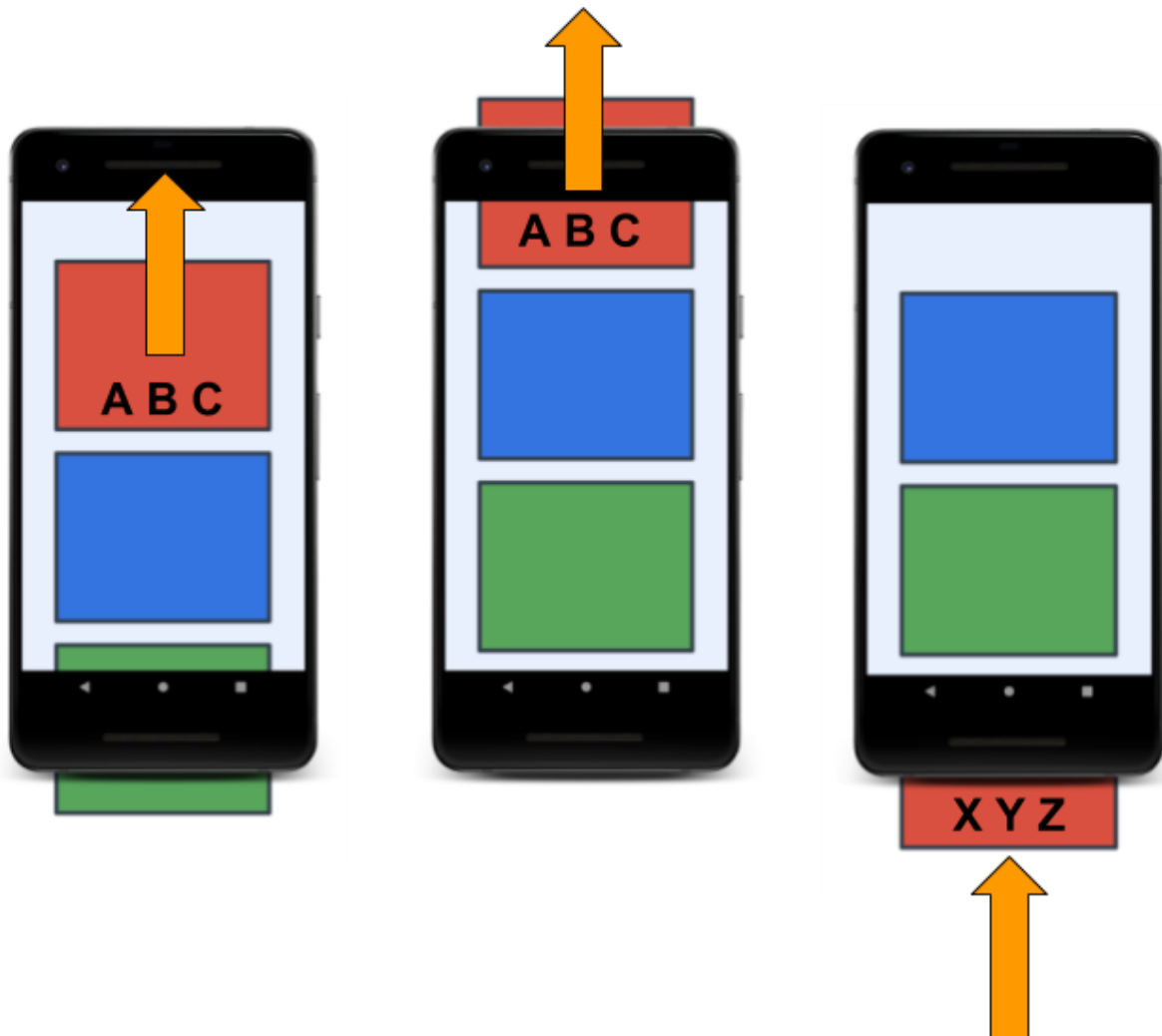
1. Before you begin

If you think about the apps you commonly use on your phone, almost every single app has at least one list. The call history screen, the contacts app, and your favorite social media app all display a list of data. As shown in the screenshot below, some of these apps display a simple list of words or phrases, where others display more complex items such as cards that include text and images. No matter what the content is, displaying a list of data is one of the most common UI tasks in Android.



To help you build apps with lists, Android provides the `RecyclerView`. `RecyclerView` is designed to be very efficient, even with large lists, by reusing, or recycling, the views that have scrolled off the screen. When a list item is scrolled off the screen, `RecyclerView` reuses that view for the next list item about to be displayed. That means, the item is filled with new content that scrolls onto the screen. This `RecyclerView` behavior saves a lot of processing time and helps lists scroll more smoothly.

In the sequence shown below, you can see that one view has been filled with data, ABC. After that view scrolls off the screen, RecyclerView reuses the view for new data, XYZ.



In this codelab, you will build the Affirmations app. Affirmations is a simple app that displays ten positive affirmations as text in a scrolling list. Then, in the follow-up codelab, you will take it a step further, add an inspiring image to each affirmation, and polish the app UI.

Prerequisites

- Create a project from a template in Android Studio.
- Add string resources to an app.
- Define a layout in XML.
- Understand classes and inheritance in Kotlin (including abstract classes).
- Inherit from an existing class and override its methods.

- Use the documentation on developer.android.com for classes provided by the Android framework.

What you'll learn

- How to use a `RecyclerView` to display a list of data.
- How to organize your code into packages
- How to use adapters with `RecyclerView` to customize how an individual list item looks.

What you'll build

- An app that displays a list of affirmation strings using a `RecyclerView`.

What you need

- A computer with Android Studio version 4.1 or higher installed.

2. Creating the project

Create an Empty Activity project

Before creating a new project, make sure that you are on Android Studio 4.1 or higher.

1. Start a new Kotlin project in Android Studio using the **Empty Activity** template.
2. Enter **Affirmations** as the app **Name**, **com.example.affirmations** as the **Package name**, and choose **API Level 19** as the **Minimum SDK**.
3. Click **Finish** to create the project.

3. Setting up the list of data

The next step in creating the Affirmations app is to add resources. You will add the following to your project.

- String resources to display as affirmations in the app.
- A source of data to provide a list of affirmations to your app.

Note: In most production Android projects, you would retrieve the affirmations data from a database or from a server. Networking and databases are beyond the scope of this codelab, so you will use a list of affirmations strings defined inside the app.

Add Affirmation strings

1. In the **Project** window, open **app > res > values > strings.xml**. This file currently has a single resource which is the name of the app.
2. In **strings.xml**, add the following affirmations as individual string resources. Name them **affirmation1**, **affirmation2**, and so on.

Affirmations text

I am strong.
I believe in myself.
Each day is a new opportunity to grow and be a better version of myself.
Every challenge in my life is an opportunity to learn from.
I have so much to be grateful for.
Good things are always coming into my life.
New opportunities await me at every turn.
I have the courage to follow my heart.
Things will unfold at precisely the right time.
I will be present in all the moments that this day brings.

The **strings.xml** file should look like this when you're done.

```
<resources>
    <string name="app_name">Affirmations</string>
    <string name="affirmation1">I am strong.</string>
    <string name="affirmation2">I believe in myself.</string>
    <string name="affirmation3">Each day is a new opportunity to grow and be
a better version of myself.</string>
    <string name="affirmation4">Every challenge in my life is an opportunity
to learn from.</string>
    <string name="affirmation5">I have so much to be grateful for.</string>
    <string name="affirmation6">Good things are always coming into my
life.</string>
    <string name="affirmation7">New opportunities await me at every
turn.</string>
    <string name="affirmation8">I have the courage to follow my
heart.</string>
    <string name="affirmation9">Things will unfold at precisely the right
time.</string>
    <string name="affirmation10">I will be present in all the moments that
this day brings.</string>
</resources>
```

Now that you have added string resources, you can reference them in your code as

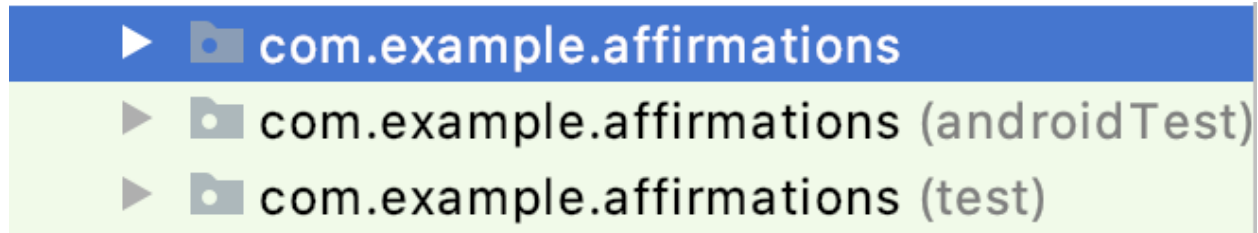
`R.string.affirmation1` or `R.string.affirmation2`.

Create a new package

Organizing your code logically helps you and other developers understand, maintain, and extend it. In the same way that you can organize paperwork into files and folders, you can organize your code into files and packages.

What is a package?

1. In Android Studio, in the **Project** window (**Android**), take a look at your new project files under **app > java** for the Affirmations app. They should look similar to the screenshot below, which shows three packages, one for your code (**com.example.affirmations**), and two for test files (**com.example.affirmations (androidTest)** and **com.example.affirmations (test)**).



2. Notice that the name of the package consists of several words separated by a period.

There are two ways in which you can make use of packages.

- Create different packages for different parts of your code. For example, developers will often separate the classes that work with data and the classes that build the UI into different packages.
- Use code from other packages in your code. In order to use the classes from other packages, you need to define them in your build system's dependencies. It's also a standard practice to `import` them in your code so you can use their shortened names (eg, `TextView`) instead of their fully-qualified names (eg, `android.widget.TextView`). For example, you have already used `import` statements for classes such as `sqrt` (`import kotlin.math.sqrt`) and `View` (`import android.view.View`).

In the Affirmations app, in addition to importing Android and Kotlin classes, you will also organize your app into several packages. Even when you don't have a lot of classes for your app, it is a good practice to use packages to group classes by functionality.

Naming packages

A package name can be anything, as long as it is globally unique; no other published package anywhere can have the same name. Because there are a very large number of packages, and coming up with random unique names is hard, programmers use conventions to make it easier to create and understand package names.

- The package name is usually structured from general to specific, with each part of the name in lowercase letters and separated by a period. Important: The period is just part of the name. It does not indicate a hierarchy in code or mandate a folder structure!
- Because internet domains are globally unique, it is a convention to use a domain, usually yours or the domain of your business, as the first part of the name.
- You can choose the names of packages to indicate what's inside the package, and how packages are related to each other.

- For code examples like this one, `com.example` followed by the name of the app is commonly used.

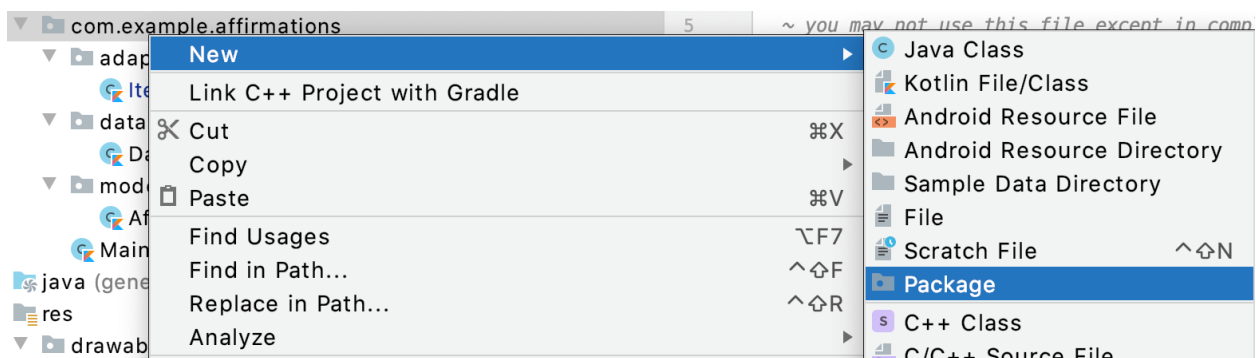
Here are some examples of predefined package names and their contents:

- `kotlin.math` - Mathematical functions and constants.
- `android.widget` - Views, such as `TextView`.

Note: While the names of packages (and their arrangement in the Android **Project** window of Android Studio as a hierarchy of folders) appear as a hierarchy, there is no actual hierarchy in the executable code. Just like the numbering system of a library categorizes and organizes books, they are still all on the same shelf, and you can take out any of them.

Create a package

1. In Android Studio, in the **Project** pane, right-click **app > java > com.example.affirmations** and select **New > Package**.



2. In the **New Package** popup, notice the suggested package name prefix. The suggested first part of the package name is the name of the package you right-clicked. While package names do not create a hierarchy of packages, reusing parts of the name is used to indicate a relationship and organization of the content!
3. In the popup, append **model** to the end of the suggested package name. Developers often use **model** as the package name for classes that model (or represent) the data.



4. Press **Enter**. This creates a new package under the **com.example.affirmations** (root) package. This new package will contain any data-related classes defined in your app.

Create the Affirmation data class

In this task, you'll create a class called `Affirmation`. An object instance of `Affirmation` represents one affirmation and contains the resource ID of the string with the affirmation.

5. Right-click on the `com.example.affirmations.model` package and select **New > Kotlin File/Class**.



6. In the popup, select **Class** and enter `Affirmation` as the name of the class. This creates a new file called `Affirmation.kt` in the `model` package.
7. Make `Affirmation` a data class by adding the `data` keyword before the class definition. This leaves you with an error, because data classes must have at least one property defined.

`Affirmation.kt`

```
package com.example.affirmations.model

data class Affirmation {
}
```

When you create an instance of `Affirmation`, you need to pass in the resource ID for the affirmation string. The resource ID is an integer.

8. Add a `val` integer parameter `stringResourceId` to the constructor of the `Affirmation` class. This gets rid of your error.

```
package com.example.affirmations.model

data class Affirmation(val stringResourceId: Int)
```

Create a class to be a data source

Data displayed in your app may come from different sources (e.g. within your app project or from an external source that requires connecting to the internet to download data). As a result, data may not be in the exact format that you need. The rest of the app should not concern itself with where the data originates from or in what format it is originally. You can and should hide away this data preparation in a separate `Datasource` class that prepares the data for the app.

Since preparing data is a separate concern, put the `Datasource` class in a separate **data** package.

1. In Android Studio, in the **Project** window, right-click **app > java > com.example.affirmations** and select **New > Package**.
2. Enter `data` as the last part of the package name.
3. Right click on the `data` package and select **new Kotlin File/Class**.
4. Enter `Datasource` as the class name.
5. Inside the `Datasource` class, create a function called `loadAffirmations()`.

The `loadAffirmations()` function needs to return a list of `Affirmations`. You do this by creating a list and populating it with an `Affirmation` instance for each resource string.

6. Declare `List<Affirmation>` as the return type of the method `loadAffirmations()`.
7. In the body of `loadAffirmations()`, add a `return` statement.
8. After the `return` keyword, call `listOf<>()` to create a `List`.
9. Inside the angle brackets `<>`, specify the type of the list items as `Affirmation`. If necessary, import `com.example.affirmations.model.Affirmation`.
10. Inside the parentheses, create an `Affirmation`, passing in `R.string.affirmation1` as the resource ID as shown below.

```
Affirmation(R.string.affirmation1)
```

11. Add the remaining `Affirmation` objects to the list of all affirmations, separated by commas. The finished code should look like the following.

```
Datasource.kt
```

```
package com.example.affirmations.data
```

```
import com.example.affirmations.R
```

```
import com.example.affirmations.model.Affirmation
```

```
class Datasource {
```

```
    fun loadAffirmations(): List<Affirmation> {
        return listOf<Affirmation>(
            Affirmation(R.string.affirmation1),
            Affirmation(R.string.affirmation2),
            Affirmation(R.string.affirmation3),
            Affirmation(R.string.affirmation4),
            Affirmation(R.string.affirmation5),
            Affirmation(R.string.affirmation6),
            Affirmation(R.string.affirmation7),
            Affirmation(R.string.affirmation8),
            Affirmation(R.string.affirmation9),
            Affirmation(R.string.affirmation10)
        )
    }
}
```

[Optional] Display the size of the Affirmations list in a TextView

To verify that you can create a list of affirmations, you can call `loadAffirmations()` and display the size of the returned list of affirmations in the `TextView` that comes with your Empty Activity app template.

1. In `layouts/activity_main.xml`, give the `TextView` that comes with your template an id of `textview`.
2. In `MainActivity` in the `onCreate()` method after the existing code, get a reference to `textview`.

```
val textView: TextView = findViewById(R.id.textview)
```

3. Then add code to create and display the size of the affirmations list. Create a `Datasource`, call `loadAffirmations()`, get the size of the returned list, convert it to a string, and assign it as the text of `textView`.

```
textView.text = Datasource().loadAffirmations().size.toString()
```

4. Run your app. The screen should look as shown below.

Affirmations

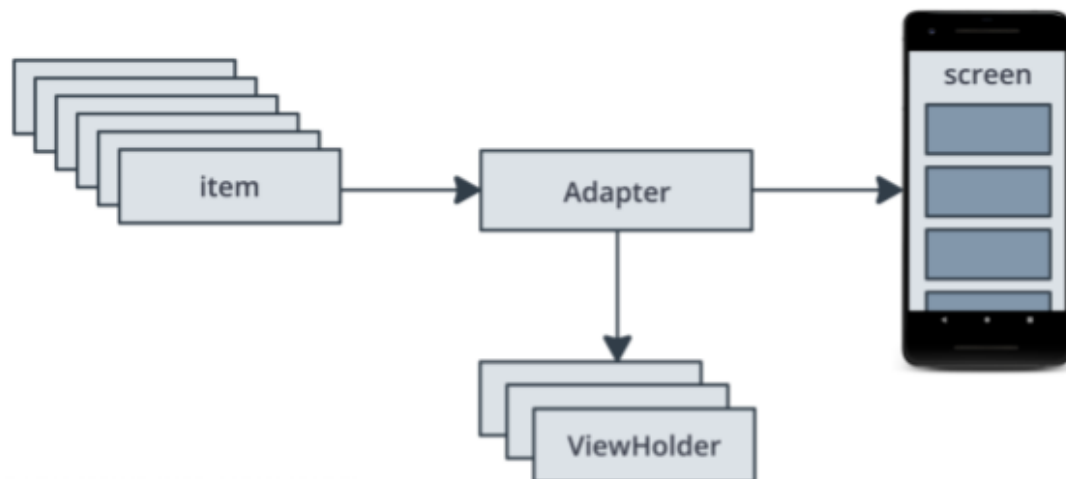
5. Delete the code you just added in `MainActivity`.

4. Adding a RecyclerView to your app

In this task, you will set up a `RecyclerView` to display the list of `Affirmations`.

There are a number of pieces involved in creating and using a `RecyclerView`. You can think of them as a division of labor. The diagram below shows an overview, and you will learn more about each piece as you implement it.

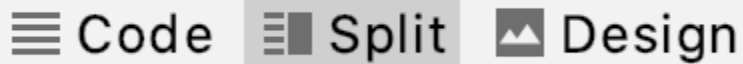
- **item** - One data item of the list to display. Represents one `Affirmation` object in your app.
- **Adapter** - Takes data and prepares it for `RecyclerView` to display.
- **ViewHolders** - A pool of views for `RecyclerView` to use and reuse to display affirmations.
- **RecyclerView** - Views on screen



Add a RecyclerView to the layout

The `Affirmations` app consists of a single activity named `MainActivity`, and its layout file is called `activity_main.xml`. First, you need to add the `RecyclerView` to the layout of the `MainActivity`.

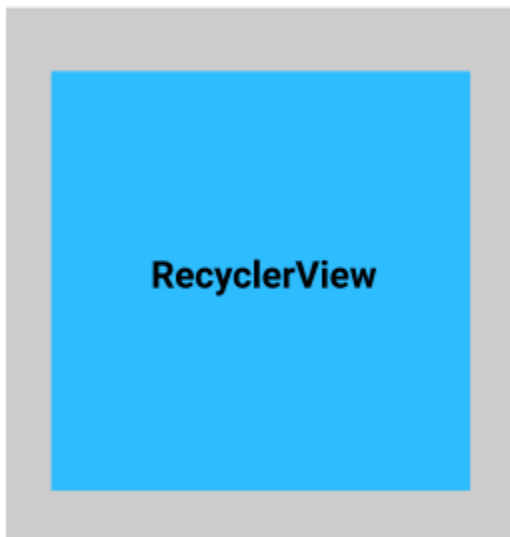
1. Open `activity_main.xml` (**app > res > layout > activity_main.xml**)
2. If you are not already using it, switch to **Split view**.



3. Delete the `TextView`.

The current layout uses `ConstraintLayout`. `ConstraintLayout` is ideal and flexible when you want to position multiple child views in a layout. Since your layout only has a single child view, `RecyclerView`, you can switch to a simpler `ViewGroup` called `FrameLayout` that should be used for holding a single child view.

FrameLayout



4. In the XML, replace `ConstraintLayout` with `FrameLayout`. The completed layout should look as shown below.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
</FrameLayout>
```

5. Switch to **Design** view.
6. In the **Palette**, select **Containers**, and find the **RecyclerView**.
7. Drag a **RecyclerView** into the layout.

8. If it appears, read the **Add Project Dependency** popup and click **OK**. (If the popup doesn't appear, no action is needed.)
9. Wait for Android Studio to finish and the `RecyclerView` to appear in your layout.
10. If necessary, change the `layout_width` and `layout_height` attributes of the `RecyclerView` to `match_parent` so that the `RecyclerView` can fill the whole screen.
11. Set the resource ID of the `RecyclerView` to `recycler_view`.

`RecyclerView` supports displaying items in different ways, such as a linear list or a grid. Arranging the items is handled by a `LayoutManager`. The Android framework provides layout managers for basic item layouts. The Affirmations app displays items as a vertical list, so you can use the `LinearLayoutManager`.

12. Switch back to **Code** view. In the XML code, inside the `RecyclerView` element, add `LinearLayoutManager` as the layout manager attribute of the `RecyclerView`, as shown below.

```
app:layoutManager="LinearLayoutManager"
```

To be able to scroll through a vertical list of items that is longer than the screen, you need to add a vertical scrollbar.

13. Inside `RecyclerView`, add an `android:scrollbars` attribute set to `vertical`.

```
android:scrollbars="vertical"
```

The final XML layout should look like the following:

```
activity_main.xml
```

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scrollbars="vertical"
        app:layoutManager="LinearLayoutManager" />

</FrameLayout>
```

14. Run your app.

The project should compile and run without any issues. However, only a white background is displayed in your app because you are missing a crucial piece of code. Right now, you have the

source of data and the `RecyclerView` added to your layout, but the `RecyclerView` has no information on how to display the `Affirmation` objects.

Implement an Adapter for the RecyclerView

Your app needs a way to take the data from `Datasource`, and format it so that each `Affirmation` can be displayed as an item in the `RecyclerView`.

Adapter is a design pattern that adapts the data into something that can be used by `RecyclerView`. In this case, you need an adapter that takes an `Affirmation` instance from the list returned by `loadAffirmations()`, and turns it into a list item view, so that it can be displayed in the `RecyclerView`.

When you run the app, `RecyclerView` uses the adapter to figure out how to display your data on screen. `RecyclerView` asks the adapter to create a new list item view for the first data item in your list. Once it has the view, it asks the adapter to provide the data to draw the item. This process repeats until the `RecyclerView` doesn't need any more views to fill the screen. If only 3 list item views fit on the screen at once, the `RecyclerView` only asks the adapter to prepare those 3 list item views (instead of all 10 list item views).

In this step, you'll build an adapter which will adapt an `Affirmation` object instance so that it can be displayed in the `RecyclerView`.

Create the Adapter

An adapter has multiple parts, and you'll be writing quite a bit of code that's more complex than what you've done in this course so far. It's okay if you don't fully understand the details at first. Once you have completed this whole app with a `RecyclerView`, you'll be able to better understand how all the parts fit together. You'll also be able to reuse this code as a base for future apps that you create with a `RecyclerView`.

Create a layout for items

Each item in the `RecyclerView` has its own layout, which you define in a separate layout file. Since you are only going to display a string, you can use a `TextView` for your item layout.

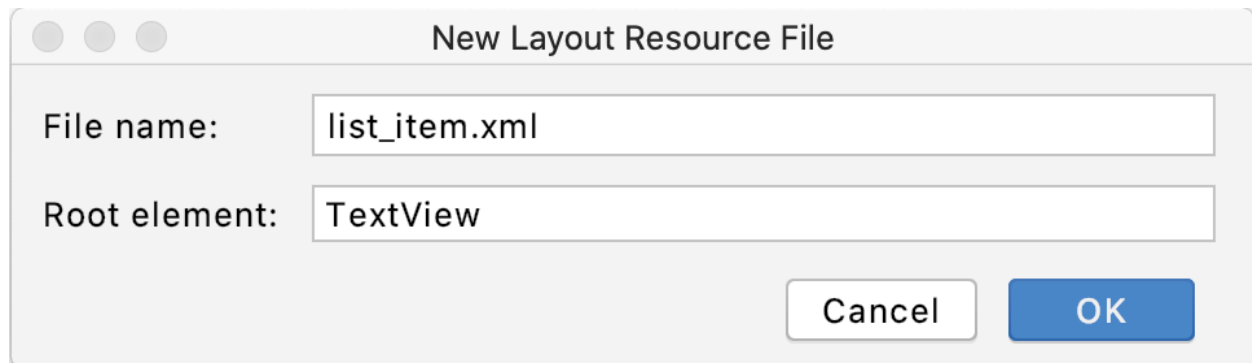
1. In **res > layout**, create a new empty **File** called `list_item.xml`.
2. Open `list_item.xml` in **Code** view.
3. Add a `TextView` with `id item_title`.
4. Add `wrap_content` for the `layout_width` and `layout_height`, as shown in the code below.

Notice that you don't need a `ViewGroup` around your layout, because this list item layout will later be inflated and added as a child to the parent `RecyclerView`.

```
list_item.xml
```

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/item_title"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

Alternatively, you could have used **File > New > Layout Resource File**, with **File name** `list_item.xml` and `TextView` as the **Root element**. Then update the generated code to match the code above.



Create an ItemAdapter class

1. In Android Studio in the **Project** pane, right-click **app > java > com.example.affirmations** and select **New > Package**.
2. Enter `adapter` as the last part of the package name.
3. Right-click on the `adapter` package and select **New > Kotlin File/Class**.
4. Enter `ItemAdapter` as the class name, finish, and the `ItemAdapter.kt` file opens.

You need to add a parameter to the constructor of `ItemAdapter`, so that you can pass the list of affirmations into the adapter.

5. Add a parameter to the `ItemAdapter` constructor that is a `val` called `dataset` of type `List<Affirmation>`. Import `Affirmation`, if necessary.
6. Since the `dataset` will be only used within this class, make it `private`.

```
ItemAdapter.kt
```

```
import com.example.affirmations.model.Affirmation

class ItemAdapter(private val dataset: List<Affirmation>) {

}
```

The `ItemAdapter` needs information on how to resolve the string resources. This, and other information about the app, is stored in a `Context` object instance that you can pass into an `ItemAdapter` instance.

7. Add a parameter to the `ItemAdapter` constructor that is a `val` called `context` of type `Context`. Position it as the first parameter in the constructor.

```
class ItemAdapter(private val context: Context, private val dataset:
List<Affirmation>) {

}
```

Create a ViewHolder

`RecyclerView` doesn't interact directly with item views, but deals with `ViewHolders` instead. A `ViewHolder` represents a single list item view in `RecyclerView`, and can be reused when possible. A `ViewHolder` instance holds references to the individual views within a list item layout (hence the name "view holder"). This makes it easier to update the list item view with new data. View holders also add information that `RecyclerView` uses to efficiently move views around the screen.

1. Inside the `ItemAdapter` class, before the closing curly brace for `ItemAdapter`, create an `ItemViewHolder` class.

```
class ItemAdapter(private val context: Context, private val dataset:
List<Affirmation>) {

    class ItemViewHolder()

}
```

- Defining a class inside another class is called creating a **nested class**.
- Since `ItemViewHolder` is only used by `ItemAdapter`, creating it inside `ItemAdapter` shows this relationship. This is not mandatory, but it helps other developers understand the structure of your program.

2. Add a private `val view` of type `View` as a parameter to the `ItemViewHolder` class constructor.
3. Make `ItemViewHolder` a subclass of `RecyclerView.ViewHolder` and pass the `view` parameter into the superclass constructor.
4. Inside `ItemViewHolder`, define a `val` property `textView` that is of type `TextView`. Assign it the view with the ID `item_title` that you defined in `list_item.xml`.

```
class ItemAdapter(private val context: Context, private val dataset:
List<Affirmation>) {

    class ItemViewHolder(private val view: View) :
RecyclerView.ViewHolder(view) {
        val textView: TextView = view.findViewById(R.id.item_title)
    }
}
```

```
}  
}
```

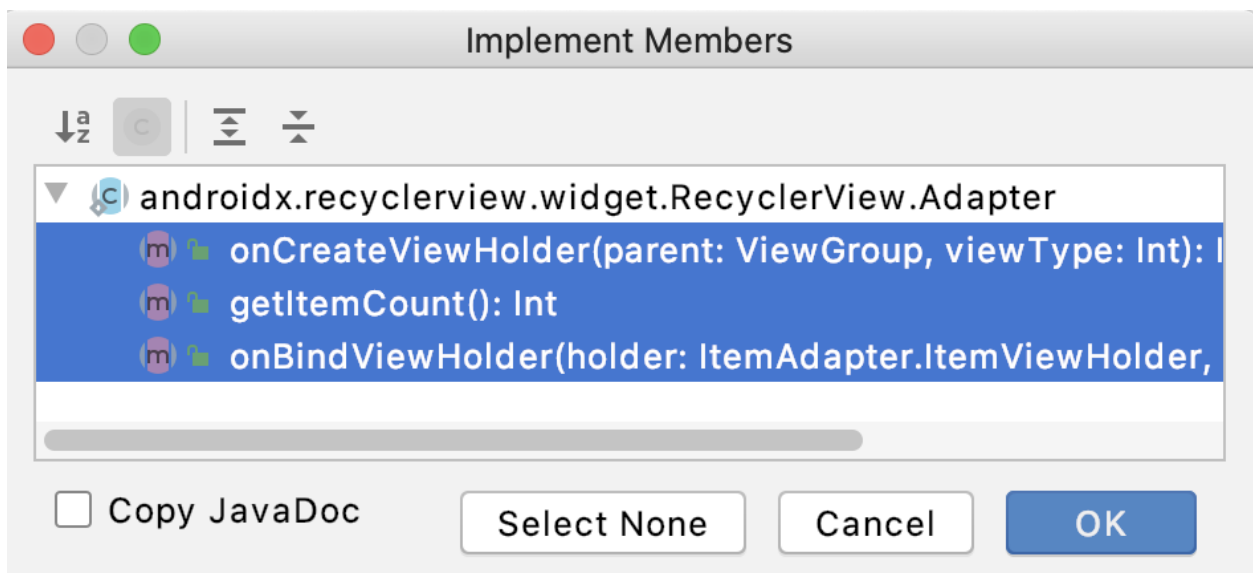
Override adapter methods

1. Add the code to extend your `ItemAdapter` from the abstract class `RecyclerView.Adapter`. Specify `ItemAdapter.ItemViewHolder` as the view holder type in angle brackets.

```
class ItemAdapter(  
    private val context: Context,  
    private val dataset: List<Affirmation>  
) : RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() {  
  
    class ItemViewHolder(private val view: View) :  
        RecyclerView.ViewHolder(view) {  
        val textView: TextView = view.findViewById(R.id.item_title)  
    }  
}
```

You will see an error because you need to implement some abstract methods from `RecyclerView.Adapter`.

2. Put your cursor on `ItemAdapter` and press **Command+I** (**Control+I** on Windows). This shows you the list of methods you need to implement: `getItemCount()`, `onCreateViewHolder()`, and `onBindViewHolder()`.



3. Select all three functions using **Shift+click** and click **OK**.

This creates stubs with the correct parameters for the three methods as shown below.

```

override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ItemViewHolder {
    TODO("Not yet implemented")
}

override fun getItemCount(): Int {
    TODO("Not yet implemented")
}

override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
    TODO("Not yet implemented")
}

```

You should see no more errors. Next you need to implement those methods so that they do the correct things for your app.

Implement getItemCount()

The `getItemCount()` method needs to return the size of your dataset. Your app's data is in the `dataset` property that you are passing into the `ItemAdapter` constructor, and you can get its size with `size`.

1. Replace `getItemCount()` with this:

```

override fun getItemCount() = dataset.size

```

It's a more concise way of writing this:

```

override fun getItemCount(): Int {
    return dataset.size
}

```

Implement onCreateViewHolder()

The `onCreateViewHolder()` method is called by the layout manager to create new view holders for the `RecyclerView` (when there are no existing view holders that can be reused). Remember that a view holder represents a single list item view.

The `onCreateViewHolder()` method takes two parameters and returns a new `ViewHolder`.

- A `parent` parameter, which is the view group that the new list item view will be attached to as a child. The parent is the `RecyclerView`.
- A `viewType` parameter which becomes important when there are multiple item view types in the same `RecyclerView`. If you have different list item layouts displayed within the `RecyclerView`, there are different item view types. You can only recycle views with the same item view type. In your case, there is only one list item layout and one item view type, so you don't have to worry about this parameter.

1. In the `onCreateViewHolder()` method, obtain an instance of `LayoutInflater` from the provided context (context of the parent). The layout inflater knows how to inflate an XML layout into a hierarchy of view objects.

```
val adapterLayout = LayoutInflater.from(parent.context)
```

2. Once you have a `LayoutInflater` object instance, add a period followed by another method call to inflate the actual list item view. Pass in the XML layout resource ID `R.layout.list_item` and the parent view group. The third boolean argument is `attachToRoot`. This argument needs to be `false`, because `RecyclerView` adds this item to the view hierarchy for you when it's time.

```
val adapterLayout = LayoutInflater.from(parent.context)
    .inflate(R.layout.list_item, parent, false)
```

Now `adapterLayout` holds a reference to the list item view (from which we can later find child views like the `TextView`).

3. In `onCreateViewHolder()`, return a new `ItemViewHolder` instance where the root view is `adapterLayout`.

```
return ItemViewHolder(adapterLayout)
```

Here is the code for `onCreateViewHolder()` so far.

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
ItemViewHolder {
    // create a new view
    val adapterLayout = LayoutInflater.from(parent.context)
        .inflate(R.layout.list_item, parent, false)

    return ItemViewHolder(adapterLayout)
}
```

Implement `onBindViewHolder()`

The last method you need to override is `onBindViewHolder()`. This method is called by the layout manager in order to replace the contents of a list item view.

The `onBindViewHolder()` method has two parameters, an `ItemViewHolder` previously created by the `onCreateViewHolder()` method, and an `int` that represents the current item position in the list. In this method, you will find the right `Affirmation` object from the data set based on position.

1. Inside `onBindViewHolder()`, create a `val item` and get the item at the given position in the dataset.

```
val item = dataset[position]
```

Finally, you need to update all the views referenced by the view holder to reflect the correct data for this item. In this case, there is only one view: the `TextView` within `ItemViewHolder`. Set the text of the `TextView` to display the `Affirmation` string for this item.

2. With an `Affirmation` object instance, you can find the corresponding string resource ID by calling `item.stringResourceId`. However, this is an integer and you need to find the mapping to the actual string value.

In the Android framework, you can call `getString()` with a string resource ID, and it will return the string value associated with it. `getString()` is a method in the `Resources` class, and you can get an instance of the `Resources` class through the `context`.

That means you can call `context.resources.getString()` and pass in a string resource ID. The resulting string can be set as the text of the `textView` in the holder `ItemViewHolder`. In short, this line of code updates the view holder to show the affirmation string.

```
holder.textView.text = context.resources.getString(item.stringResourceId)
```

The completed `onBindViewHolder()` method should look like this.

```
override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {  
    val item = dataset[position]  
    holder.textView.text  
= context.resources.getString(item.stringResourceId)  
}
```

Here is the finished adapter code.

ItemAdapter.kt

```
package com.example.affirmations.adapter  
  
import android.content.Context  
import android.view.LayoutInflater  
import android.view.View  
import android.view.ViewGroup  
import android.widget.TextView  
import androidx.recyclerview.widget.RecyclerView  
import com.example.affirmations.R  
import com.example.affirmations.model.Affirmation  
  
/**  
 * Adapter for the [RecyclerView] in [MainActivity]. Displays [Affirmation]  
 data object.  
 */  
class ItemAdapter(  
    private val context: Context,  
    private val dataset: List<Affirmation>  
) : RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() {
```

```

    // Provide a reference to the views for each data item
    // Complex data items may need more than one view per item, and
    // you provide access to all the views for a data item in a view holder.
    // Each data item is just an Affirmation object.
    class ItemViewHolder(private val view: View) :
        RecyclerView.ViewHolder(view) {
        val textView: TextView = view.findViewById(R.id.item_title)
    }

    /**
     * Create new views (invoked by the layout manager)
     */
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        ItemViewHolder {
        // create a new view
        val adapterLayout = LayoutInflater.from(parent.context)
            .inflate(R.layout.list_item, parent, false)

        return ItemViewHolder(adapterLayout)
    }

    /**
     * Replace the contents of a view (invoked by the layout manager)
     */
    override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
        val item = dataset[position]
        holder.textView.text =
            context.resources.getString(item.stringResourceId)
    }

    /**
     * Return the size of your dataset (invoked by the layout manager)
     */
    override fun getItemCount() = dataset.size
}

```

Now that you've implemented the `ItemAdapter`, you need to tell the `RecyclerView` to use this adapter.

Modify the MainActivity to use a RecyclerView

To finish, you need to use your `Datasource` and `ItemAdapter` classes to create and display items in the `RecyclerView`. You do this in `MainActivity`.

1. Open `MainActivity.kt`.
2. In `MainActivity`, go to the `onCreate()` method. Insert the new code described in the following steps after the call to `setContentView(R.layout.activity_main)`.
3. Create an instance of `Datasource`, and call the `loadAffirmations()` method on it. Store the returned list of affirmations in a `val` named `myDataset`.

```
val myDataset = Datasource().loadAffirmations()
```

4. Create a variable called `recyclerView` and use `findViewById()` to find a reference to the `RecyclerView` within the layout.

```
val recyclerView = findViewById<RecyclerView>(R.id.recycler_view)
```

5. To tell the `recyclerView` to use the `ItemAdapter` class you created, create a new `ItemAdapter` instance. `ItemAdapter` expects two parameters: the context (`this`) of this activity, and the affirmations in `myDataset`.
6. Assign the `ItemAdapter` object to the `adapter` property of the `recyclerView`.

```
recyclerView.adapter = ItemAdapter(this, myDataset)
```

7. Since the layout size of your `RecyclerView` is fixed in the activity layout, you can set the `setHasFixedSize` parameter of the `RecyclerView` to `true`. This setting is only needed to improve performance. Use this setting if you know that changes in content do not change the layout size of the `RecyclerView`.

```
recyclerView.setHasFixedSize(true)
```

8. When you are done, the code for `MainActivity` should be similar to the following.

`MainActivity.kt`

```
package com.example.affirmations

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.RecyclerView
import com.example.affirmations.adapter.ItemAdapter
import com.example.affirmations.data.Datasource

class MainActivity : AppCompatActivity() {

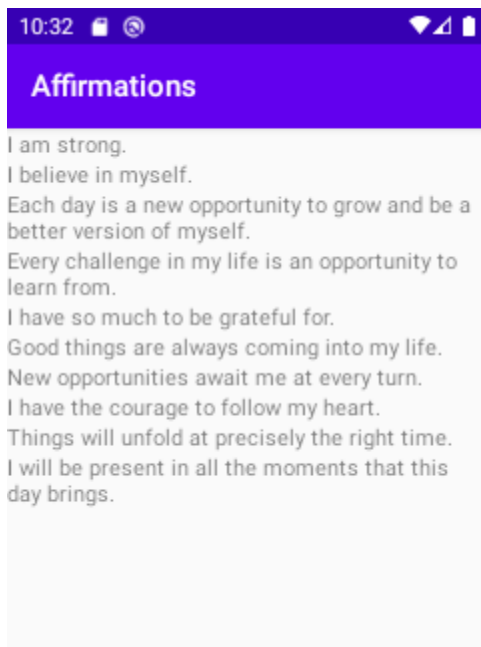
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Initialize data.
        val myDataset = Datasource().loadAffirmations()

        val recyclerView = findViewById<RecyclerView>(R.id.recycler_view)
        recyclerView.adapter = ItemAdapter(this, myDataset)

        // Use this setting to improve performance if you know that changes
        // in content do not change the layout size of the RecyclerView
        recyclerView.setHasFixedSize(true)
    }
}
```

9. Run your app. You should see a list of affirmation strings displayed on screen.



Congratulations! You've just created an app that displays a list of data with `RecyclerView` and a custom adapter. Take some time to look over the code you created, and understand how the different pieces work together.

This app has all the required pieces to display your affirmations, but it's not quite ready for production. The UI could use some improvement. In the next codelab, you'll improve your code, learn how to add images to the app, and polish the UI.

5. Solution code

The solution code for this codelab is in the project and module shown below. Note that some of the Kotlin files are in different packages, as indicated by the `package` statement at the start of the file.

res/values/strings.xml

```
<resources>
    <string name="app_name">Affirmations</string>
    <string name="affirmation1">I am strong.</string>
    <string name="affirmation2">I believe in myself.</string>
    <string name="affirmation3">Each day is a new opportunity to grow and be
a better version of myself.</string>
    <string name="affirmation4">Every challenge in my life is an opportunity
to learn from.</string>
    <string name="affirmation5">I have so much to be grateful for.</string>
    <string name="affirmation6">Good things are always coming into my
life.</string>
    <string name="affirmation7">New opportunities await me at every
turn.</string>
    <string name="affirmation8">I have the courage to follow my
```

```
heart.</string>
    <string name="affirmation9">Things will unfold at precisely the right
time.</string>
    <string name="affirmation10">I will be present in all the moments that
this day brings.</string>
</resources>
```

affirmations/data/Datasource.kt

```
package com.example.affirmations.data
```

```
import com.example.affirmations.R
import com.example.affirmations.model.Affirmation
```

```
class Datasource {

    fun loadAffirmations(): List<Affirmation> {
        return listOf<Affirmation>(
            Affirmation(R.string.affirmation1),
            Affirmation(R.string.affirmation2),
            Affirmation(R.string.affirmation3),
            Affirmation(R.string.affirmation4),
            Affirmation(R.string.affirmation5),
            Affirmation(R.string.affirmation6),
            Affirmation(R.string.affirmation7),
            Affirmation(R.string.affirmation8),
            Affirmation(R.string.affirmation9),
            Affirmation(R.string.affirmation10)
        )
    }
}
```

affirmations/model/Affirmation.kt

```
package com.example.affirmations.model
```

```
data class Affirmation(val stringResourceId: Int)
```

affirmations/MainActivity.kt

```
package com.example.affirmations
```

```
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.RecyclerView
import com.example.affirmations.adapter.ItemAdapter
import com.example.affirmations.data.Datasource
```

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

```

        // Initialize data.
        val myDataset = Datasource().loadAffirmations()

        val recyclerView = findViewById<RecyclerView>(R.id.recycler_view)
        recyclerView.adapter = ItemAdapter(this, myDataset)

        // Use this setting to improve performance if you know that changes
        // in content do not change the layout size of the RecyclerView
        recyclerView.setHasFixedSize(true)
    }
}

```

affirmations/adapters/ItemAdapter.kt

```

package com.example.affirmations.adapters

import android.content.Context
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView
import com.example.affirmations.R
import com.example.affirmations.model.Affirmation

/**
 * Adapter for the [RecyclerView] in [MainActivity]. Displays [Affirmation]
 * data object.
 */
class ItemAdapter(
    private val context: Context,
    private val dataset: List<Affirmation>
) : RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() {

    // Provide a reference to the views for each data item
    // Complex data items may need more than one view per item, and
    // you provide access to all the views for a data item in a view holder.
    // Each data item is just an Affirmation object.
    class ItemViewHolder(private val view: View) :
        RecyclerView.ViewHolder(view) {
        val textView: TextView = view.findViewById(R.id.item_title)
    }

    /**
     * Create new views (invoked by the layout manager)
     */
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        ItemViewHolder {
        // create a new view
        val adapterLayout = LayoutInflater.from(parent.context)
            .inflate(R.layout.list_item, parent, false)

        return ItemViewHolder(adapterLayout)
    }

    /**

```

```

        * Replace the contents of a view (invoked by the layout manager)
        */
        override fun onBindViewHolder(holder: ItemViewHolder, position: Int) {
            val item = dataset[position]
            holder.textView.text =
context.resources.getString(item.stringResourceId)
        }

/**
 * Return the size of your dataset (invoked by the layout manager)
 */
        override fun getItemCount() = dataset.size
    }
}

```

src/main/res/layout/activty_main.xml

```

<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scrollbars="vertical"
        app:layoutManager="LinearLayoutManager" />

</FrameLayout>

```

src/main/res/layout/list_item.xml

```

<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/item_title"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />

```

6. Summary

- RecyclerView widget helps you display a list of data.
- RecyclerView uses the adapter pattern to adapt and display the data.
- ViewHolder creates and holds the views for RecyclerView.
- RecyclerView comes with built in LayoutManagers. RecyclerView delegates how items are laid out to LayoutManagers.

To implement the adapter:

- Create a new class for the adapter, for example, ItemAdapter.

- Create a custom `ViewHolder` class that represents a single list item view. Extend from `RecyclerView.ViewHolder` class.
- Modify the `ItemAdapter` class to extend from the `RecyclerView.Adapter` class with the custom `ViewHolder` class.
- Implement these methods within the adapter: `getItemCount()`, `onCreateViewHolder()`, and `onBindViewHolder()`.

7. Learn more

- [Create a list with RecyclerView](#)
- [RecyclerView class](#)
- [RecyclerView.Adapter](#)
- [RecyclerView.ViewHolder](#)
- [RecyclerView library](#)
- [Lists](#) in Material Design
- [Enhance your UI with MaterialCardView and images](#)